

```
In [1]: # Behzad Ghabaei
# CS 82A - Data Science
# Module 3 Lab 1: Rescue a Messy Dataset
# module3_lab.ipynb
# Instructor Seno
# 9/15/2026
```

Step 1: Load and size up the file

```
In [2]: import pandas as pd
df = pd.read_csv("messy_sales.csv")
```

The 4 commands you run on a new dataset are: **df.head()** **df.shape** **df.info()** **df.describe()**. Next, display the shape, first ten rows, and column types. When running **df.shape** we see (300, 6). We see zip is int64 and date is object, and both are wrong.

```
In [3]: df.shape
```

```
Out[3]: (300, 6)
```

The first 10 rows displayed

```
In [4]: df.head(10)
```

```
Out[4]:
```

	order_id	date	product	price	qty	zip
0	1254	04/06/2026	mug	11.36	4	60614
1	1057	2026-05-30	charger	14.79	3	30303
2	1150	05/06/2026	mug	5.50	1	10001
3	1066	05/30/2026	notebook	NaN	1	98101
4	1114	04/06/2026	webcam	45.59	4	90405
5	1280	04/22/2026	pen set	NaN	4	30303
6	1204	04/18/2026	charger	8.41	2	2134
7	1249	2026-05-15	keyboard	75.04	4	90405
8	1037	2026-05-19	mug	9.31	2	60614
9	1177	2026-05-13	mug	10.11	3	90210

We see below that zip says int64, and date is str, we will be adjusting these two types.

```
In [5]: df.dtypes
```

```
Out[5]: order_id    int64
date             str
product          str
price           float64
qty             int64
zip             int64
dtype: object
```

We see that the non-null count of "price" is 288, the rest are 300

```
In [6]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   order_id    300 non-null    int64
1   date        300 non-null    str
2   product     300 non-null    str
3   price       288 non-null    float64
4   qty         300 non-null    int64
5   zip         300 non-null    int64
dtypes: float64(1), int64(3), str(2)
memory usage: 14.2 KB
```

We can observe the mean and median of the following columns. Notice that date is not there, but zip is there.

```
In [7]: df.describe()
```

```
Out[7]:
```

	order_id	price	qty	zip
count	300.000000	288.000000	300.000000	300.000000
mean	1144.816667	56.068438	2.913333	45385.276667
std	84.515319	72.307182	1.560107	39345.608641
min	1000.000000	2.930000	-5.000000	2116.000000
25%	1071.750000	10.945000	2.000000	2134.000000
50%	1144.500000	37.530000	3.000000	30303.000000
75%	1217.250000	61.417500	4.000000	90210.000000
max	1291.000000	479.290000	5.000000	98101.000000

These are all the columns

```
In [8]: df.columns
```

```
Out[8]: Index(['order_id', 'date', 'product', 'price', 'qty', 'zip'], dtype='str')
```

Step 2: Write your first observations (markdown cell)

Three problems or more, that I can already see in this dataset are described in the **data dictionary**. The order_id are integers instead of string types.

The dates are recorded in two different formats. Some have the year first with a hyphen, and some have slashes with the year last. MM/DD/YYYY or YYYY-MM-DD

The "price" is missing for (300 - 288 = 12) 12 values, which can be seen with: df.isna().sum(). Missing cells known as NaN are listed by price, (this returns 12 missing values.)

A decision has to be made on the meaning of negative quantity values. Are they returns, and should they be turned to 0, or should they be turned to positive numbers, due to a sign flipping error? I think that taking the absolute value of each negative, would be the best choice, and according to our textbook, "Hands on Intro to Data Science Using

Python" Shah describes that changing the sign to positive is sufficient to smooth noisy data, and remedy faulty negative entries. (Chapter 2, "Data Cleaning")

Lastly, the zip code should be in a string format to prevent leading zeros from getting lost, from Tutorial 4: Types carry meaning: codes are text, not numbers: `df["zip"] = df["zip"].astype(str).str.zfill(5)`

Step 3: Count the missing values

The results show 12 missing, under price.

```
In [9]: # apply tutorial 1 to the whole dataframe
df.isna().sum() # 12 missing values under price
```

```
Out[9]: order_id    0
       date        0
       product     0
       price      12
       qty         0
       zip         0
       dtype: int64
```

Step 4: Impute the missing prices

We'll be using the median for the missing values of price. This is much better than the average because it represents the 50 percentile of the data. This value is 37.5. Since the data only has 12 missing values, it might be better to put them in, one by one, by hand, instead of using a set price. This is because the merchandise ranges from notebooks and pen sets, to monitors and chargers, which are considerably more expensive. However, we will use the median instead of the mean.

```
In [10]: df.shape # always run shape before and after imputing missing prices.
```

```
Out[10]: (300, 6)
```

```
In [11]: # Fill the missing prices using the Tutorial 2 pattern
# Looking for: Impossible Values, Suspicious Outliers, Missing Values, Floating Point Noise
fill_value = df["price"].median() # The 50% median of price when running df.describe() is 37.53
# whether mean or median fits your lab data better
df["price"] = df["price"].fillna(fill_value)
print(df.head(10))
```

	order_id	date	product	price	qty	zip
0	1254	04/06/2026	mug	11.36	4	60614
1	1057	2026-05-30	charger	14.79	3	30303
2	1150	05/06/2026	mug	5.50	1	10001
3	1066	05/30/2026	notebook	37.53	1	98101
4	1114	04/06/2026	webcam	45.59	4	90405
5	1280	04/22/2026	pen set	37.53	4	30303
6	1204	04/18/2026	charger	8.41	2	2134
7	1249	2026-05-15	keyboard	75.04	4	90405
8	1037	2026-05-19	mug	9.31	2	60614
9	1177	2026-05-13	mug	10.11	3	90210

```
In [12]: df.shape
```

```
Out[12]: (300, 6)
```

Price no longer has a null value, as shown below, 0 missing prices afterwards

```
In [13]: df.isna().sum() # df.isnull().sum() - This makes price ---- 0
```

```
Out[13]: order_id    0
         date        0
         product    0
         price      0
         qty        0
         zip        0
         dtype: int64
```

Step 5: Remove duplicate rows

According to `duplicated().sum()` functions there are 8 repeated values which can skew our data. By removing these 8, the data can become more accurate, and the average will not be pulled away from the actual. These extra 8 duplicates can be the cause of inaccurate results and suspicious outliers.

```
In [14]: # Count duplicates, remove them, and confirm the new shape (Tutorial 3).
         #8 duplicates found; df.shape (292, 6).
         df.shape
```

```
Out[14]: (300, 6)
```

```
In [15]: print(df.duplicated().sum()) # counts fully repeated rows
         df = df.drop_duplicates()    # removes them.
```

8

The new shape reveals 292, and recorded in the Cleaning Log.

```
In [16]: df.shape # 300 - 8 = 292
```

```
Out[16]: (292, 6)
```

Step 6: Repair the zip codes

Recall that zip was int64, we want to convert the zip to text or str. This will restore the leading zeros.

```
In [17]: # Convert zip to text and restore the leading zeros using Tutorial 4 adapted to 5-character zips.
         df["zip"] = df["zip"].astype(str).str.zfill(5) # zips like 02134 display with their leading zero.
```

```
In [18]: print(df.head(7)) # Leading zeros are visible on "zip"
```

	order_id	date	product	price	qty	zip
0	1254	04/06/2026	mug	11.36	4	60614
1	1057	2026-05-30	charger	14.79	3	30303
2	1150	05/06/2026	mug	5.50	1	10001
3	1066	05/30/2026	notebook	37.53	1	98101
4	1114	04/06/2026	webcam	45.59	4	90405
5	1280	04/22/2026	pen set	37.53	4	30303
6	1204	04/18/2026	charger	8.41	2	02134

Step 7: Standardize the dates

Previously, date was an str type. The date can be converted to `datetime64[]`. The min and max can be obtained this way from the `describe()` function.

```
In [19]: # Convert the date column to real datetimes (Tutorial 5) and verify the dtype.
# datetime64[ns] in the type list.
df["date"] = pd.to_datetime(df["date"], format="mixed")
```

Below datetime64 is listed for date. Dates are not sorted but formatted the same.

```
In [20]: df.dtypes #df["date"] also can verify the dtype, we see data type "datetime64" across from "date"
```

```
Out[20]: order_id      int64
date      datetime64[us]
product    str
price      float64
qty        int64
zip        str
dtype: object
```

```
In [21]: df["date"]
```

```
Out[21]: 0      2026-04-06
1      2026-05-30
2      2026-05-06
3      2026-05-30
4      2026-04-06
...
294    2026-04-21
296    2026-04-27
297    2026-04-27
298    2026-04-25
299    2026-05-16
Name: date, Length: 292, dtype: datetime64[us]
```

Step 8: Investigate the negative quantities

The negative quantities could be faulty entries, and taking the absolute value retains the accuracy of the data. They should be addressed by using the absolute value or by fixing them by hand. However absolute value is more efficient.

```
In [22]: # Use the filtering pattern (Tutorial 6) to display rows where qty is below zero
# Decide: returns to flag, typos to correct, or rows to remove. Apply your choice in code.
df[df["qty"] < 0]
```

```
Out[22]:
```

	order_id	date	product	price	qty	zip
202	1140	2026-04-18	webcam	38.69	-5	98101
262	1233	2026-05-09	desk lamp	22.64	-4	10001
297	1025	2026-04-27	keyboard	47.30	-2	02116

```
In [23]: # absolute values of each can be taken
# Use absolute value to correct negative quantities, from the math library
df["qty"] = df["qty"].abs()
```

Step 9: Save the cleaned file

A new csv file has been created with our Data Pre Processing:

1. Data Cleaning

- Smooth noisy data
- Handling missing data

c. Data Wrangling

2. Data Integration
3. Data Transformation
4. Data Reduction
5. Data Discretization

```
In [24]: # sales_clean.csv
# without the index column.
# Hint: to_csv has an index= option.
# Save the cleaned DataFrame to a CSV file
df.to_csv('sales_clean.csv', index=False)
```

Step 10: Write the cleaning log (markdown cell)

Under the title Cleaning Log, record every change: what, why, and row counts (300 loaded; 12 prices filled with ____; 8 duplicates removed leaving 292; zips restored; dates standardized; negatives handled how and why). End with one sentence on how a wrong choice here could mislead a business decision.

Cleaning Log

In the 10 steps, the following things were done to the data. The recorded changes.

1. The data was loaded with pandas as "messy_data.csv" 300 row counts were loaded initially from the csv file.
2. Data Dictionary mentions warnings and precautions to take.
3. Missing values under "price" were located. 12 values missing, and NaN was used before.
4. We used the median at 37.53 to resolve the problem of price missing. The median filled these blank spaces. Although missing prices were listed under notebook and pen set, other items such as webcam and monitor, desk lamp, and keyboards balances the expenditure. The median was better to use than the mean, which was much higher at 56.0. The 50 percentile best represented the price data.
5. Duplicated rows (8) were removed which made the new shape to 292. Duplicates can skew the data and make the output less precise. When data is duplicated it can corrupt statistics and make data noise with outliers that are not accurate.
6. The zip codes were changed from integers to string. This would preserve the accuracy of each. In order to preserve the leading zero, the data type has to be changed to str. This will fix the geographic zip code structure.
7. The dates were standardized to datetime64[] this is the data type it was changed to. When using the describe() function or sorting the dates, these can be organized from least to greatest. The timestamps were conflicting before this. Some were MM/DD/YYYY, while others were YYYY-MM-DD. Now they all are the same as YYYY-MM-DD. i.e.:
df.sort_values("date")
8. The negative quantities under "qty" were changed from negative to their absolute value of the negative value. i.e. -3 becomes +3. This means that the merchandise was not returned but is still in use. This will smooth noisy data, and remedy faulty entries. The negatives are handled with the absolute values function to make each impossible value

into a true value. It's impossible to have a negative quantity, which is the same as a zero, so this must be a typo, where the sign is flipped by mistake.

9. The new csv data was saved to "sales_clean.csv"

Conclusion: If the negative values were used, the overall metrics could become skewed and inaccurate, causing a lack of accuracy on the entire statistical reports made. This would build suspicious outliers and would be misleading. Impossible values that do not make sense should always be addressed to prevent data corruption. If the zip codes were not properly preserved, the outcome could be a confused marketing report on the geographic location of company products as well as distortion on the min, max, median, and mean descriptions of the data.

These prices were previously NaN, now they are represented with the median.

```
In [25]: df[df["price"] == 37.53 ]
```

```
Out[25]:
```

	order_id	date	product	price	qty	zip
3	1066	2026-05-30	notebook	37.53	1	98101
5	1280	2026-04-22	pen set	37.53	4	30303
30	1088	2026-04-28	pen set	37.53	2	02134
44	1101	2026-05-12	keyboard	37.53	3	02116
49	1199	2026-04-18	charger	37.53	5	30303
50	1018	2026-05-04	monitor	37.53	4	10001
55	1170	2026-04-04	monitor	37.53	1	02116
91	1040	2026-05-08	pen set	37.53	3	02134
104	1240	2026-04-16	headphones	37.53	1	60614
128	1261	2026-05-09	desk lamp	37.53	2	90405
178	1005	2026-05-24	monitor	37.53	2	02134
251	1155	2026-04-08	charger	37.53	3	02134

negative qty no longer exists

```
In [26]: df[df["qty"] < 0]
```

```
Out[26]:
```

order_id	date	product	price	qty	zip
----------	------	---------	-------	-----	-----

Date and zip have been adjusted with pre processing and cleaning

```
In [27]: df["date"].head()
```

```
Out[27]:
```

0	2026-04-06
1	2026-05-30
2	2026-05-06
3	2026-05-30
4	2026-04-06

Name: date, dtype: datetime64[us]

```
In [28]: df["zip"].head(7)
```

```
Out[28]: 0    60614
         1    30303
         2    10001
         3    98101
         4    90405
         5    30303
         6    02134
         Name: zip, dtype: str
```

```
In [29]: ##### The new count is 292, the new qty min is 1 and the new qmax is 5
```

```
In [30]: df.describe() #product and zip are not included
```

```
Out[30]:
```

	order_id	date	price	qty
count	292.000000	292	292.000000	292.000000
mean	1145.500000	2026-05-01 16:50:57.534246	55.523151	2.986301
min	1000.000000	2026-04-01 00:00:00	2.930000	1.000000
25%	1072.750000	2026-04-18 00:00:00	11.045000	2.000000
50%	1145.500000	2026-05-02 00:00:00	37.530000	3.000000
75%	1218.250000	2026-05-15 00:00:00	55.510000	4.000000
max	1291.000000	2026-05-30 00:00:00	479.290000	5.000000
std	84.437354	NaN	71.767085	1.409279

We see that dates can be sorted now

```
In [31]: df.sort_values("date")
```

```
Out[31]:
```

	order_id	date	product	price	qty	zip
223	1076	2026-04-01	mug	8.67	1	30303
181	1116	2026-04-01	pen set	13.49	1	60614
240	1127	2026-04-01	pen set	14.33	2	02116
273	1038	2026-04-02	headphones	35.57	1	98101
56	1230	2026-04-02	desk lamp	26.40	3	90405
...	...	...	...	...	...	...
188	1026	2026-05-30	pen set	9.51	3	02116
42	1288	2026-05-30	monitor	348.68	1	10001
244	1229	2026-05-30	keyboard	37.48	5	02116
20	1091	2026-05-30	keyboard	50.90	2	60614
113	1198	2026-05-30	keyboard	69.65	4	98101

292 rows × 6 columns

End Of Lab - Behzad Ghabaei

```
In [ ]:
```